

	SR UNIVERSITY	
	Campus Warangal	
	Program: II - B. Tech (CS& AI) Batch:	
	Faculty's: Dr. Mathivanan	
	Department: Computer Science and AI	Semester: 3
AI Assisted Coding-Assignment 07 Date: 19/9/2026		
Name -Himanshu Kumar Roll no-2503A54045 Batch-58		

Q1. Class with Conditional Logic – Employee Salary

Scenario:

An AI coding assistant is used to develop a simple employee salary management program.

Task:

Using AI-assisted code completion, write a Python class Employee that:

- Stores employee name and basic salary.
- Has a method calculate_bonus() that:
 - Returns 10% of salary if salary is ₹50,000 or above.
 - Returns 5% of salary otherwise.
- Has a method display_details() to display the employee name, salary, and bonus.
- Create at least two employee objects and test the class.

Prompt:

Generate a complete, executable Python program for an Employee Salary Calculator using Object-Oriented Programming. Requirements: Create a class named Employee with attributes name and basic_salary. Add a method calculate_bonus() that returns a 10% bonus if basic_salary >= 50000, otherwise 5%. Add a method calculate_total_salary() returning

basic_salary + bonus. Include a display_details() method showing Name, Basic Salary, Bonus, and Total Salary. Instantiate objects testing salaries below, equal to, and above ₹50,000. Use brief inline comments and clean standard formatting.

Python Code

```
Assignment07.py > ...
1  class Employee:
2      def __init__(self, name: str, basic_salary: float):
3          self.name = name
4          self.basic_salary = basic_salary
5
6      def calculate_bonus(self) -> float:
7          """Calculates bonus based on basic salary threshold of 50,000."""
8          if self.basic_salary >= 50000:
9              return self.basic_salary * 0.10
10         else:
11             return self.basic_salary * 0.05
12
13     def calculate_total_salary(self) -> float:
14         """Returns total salary including bonus."""
15         return self.basic_salary + self.calculate_bonus()
16
17     def display_details(self):
18         """Displays formatted employee details."""
19         bonus = self.calculate_bonus()
20         total = self.calculate_total_salary()
21         print(f"Employee Name : {self.name}")
22         print(f"Basic Salary   : ₹{self.basic_salary:,.2f}")
23         print(f"Bonus           : ₹{bonus:,.2f}")
24         print(f"Total Salary    : ₹{total:,.2f}")
25         print("-" * 35)
26
27
```

```

25         print("-" * 35)
26
27
28     # Test Cases
29     if __name__ == "__main__":
30         # Test Case 1: Salary below 50,000
31         emp1 = Employee("Rahul", 35000)
32
33         # Test Case 2: Salary exactly 50,000 (Boundary condition)
34         emp2 = Employee("Priya", 50000)
35
36         # Test Case 3: Salary above 50,000
37         emp3 = Employee("Karan", 75000)
38
39         # Display results
40         emp1.display_details()
41         emp2.display_details()
42         emp3.display_details()

```

Output:

```

Employee Name : Rahul
Basic Salary  : ₹35,000.00
Bonus         : ₹1,750.00
Total Salary  : ₹36,750.00
-----
Employee Name : Priya
Basic Salary  : ₹50,000.00
Bonus         : ₹5,000.00
Total Salary  : ₹55,000.00
-----
Employee Name : Karan
Basic Salary  : ₹75,000.00
Bonus         : ₹7,500.00
Total Salary  : ₹82,500.00
-----

```

Observation:

❑ Key Finding: The threshold condition (salary >= 50000) correctly applies a 10% bonus for salaries ≥ ₹50,000 and 5% for lower values.

❑ Boundary Check: Exactly ₹50,000 correctly triggers the 10% tier

Q2. Loop and Conditional Logic – Student Marks

Scenario:

A teacher wants to analyze the marks of students using an AI coding assistant.

Task:

Using AI-assisted code completion, write a Python program that:

- Accepts a list of student marks.
- Uses a loop to process each mark.
- Displays:
 - "Distinction" for marks ≥ 75
 - "Pass" for marks ≥ 40 and < 75
 - "Fail" for marks < 40
- Ignores invalid marks outside 0–100.
- Calculates and displays the number of students in each category.
- Test the program with a suitable list of marks.

Prompt:

Write a Python script to process a list of student marks using loops and conditional logic. Categorize marks as follows: "Distinction" (≥ 75), "Pass" (40 to 74), and "Fail" (< 40). Ignore invalid marks outside the 0–100 range. Keep track of and display the total counts for each category. Include a test list containing valid and invalid marks.

Code:

```

45 def analyze_student_marks(marks_list: list):
46     """Processes student marks, categorizes them, and tracks category counts."""
47     distinction_count = 0
48     pass_count = 0
49     fail_count = 0
50
51     print("--- Individual Mark Analysis ---")
52     for mark in marks_list:
53         # Check for invalid range
54         if mark < 0 or mark > 100:
55             print(f"Mark: {mark:<5} -> Ignored (Invalid range 0-100)")
56             continue
57
58         # Categorize valid marks
59         if mark >= 75:
60             category = "Distinction"
61             distinction_count += 1
62         elif mark >= 40:
63             category = "Pass"
64             pass_count += 1
65         else:
66             category = "Fail"
67             fail_count += 1
68
69         print(f"Mark: {mark:<5} -> {category}")
70
71     # Summary
72     print("\n--- Summary Category Counts ---")
73     print(f"Distinction : {distinction_count}")
74     print(f"Pass : {pass_count}")
75     print(f"Fail : {fail_count}")
76
77

```

```

# Test data with valid, boundary, and invalid marks
if __name__ == "__main__":
    sample_marks = [85, 38, 75, 105, 40, 62, -15, 20]
    analyze_student_marks(sample_marks)

```

Output:

```
--- Individual Mark Analysis ---
Mark: 85    -> Distinction
Mark: 38    -> Fail
Mark: 75    -> Distinction
Mark: 105   -> Ignored (Invalid range 0-100)
Mark: 40    -> Pass
Mark: 62    -> Pass
Mark: -15   -> Ignored (Invalid range 0-100)
Mark: 20    -> Fail

--- Summary Category Counts ---
Distinction : 2
Pass        : 2
Fail        : 2
```

Observation:

- ❑ **Key Finding:** Input validation skips out-of-range values (< 0 or > 100) using `continue`.
- ❑ **Classification:** Valid marks are correctly categorized into Distinction (≥ 75), Pass ($40 - 74$), and Fail (< 40) with accurate category counts.

Q3. Class + Loop – Product Inventory

Scenario:

A small store wants to maintain product inventory using a Python program generated with AI assistance.

Task:

Using AI-assisted code completion, create a class `Product` that:

- Stores product name, price, and quantity.
- Has a method `total_value()` that calculates $\text{price} \times \text{quantity}$.
- Create a list of multiple `Product` objects.
- Use a loop to calculate the total inventory value.
- Display the products whose quantity is less than 5 as "Low Stock".
- Test the program with at least three products.

Prompt:

Create a Python program with a Product class having name, price, and quantity attributes, along with a total_value() method. Create a list of multiple Product objects, iterate through them using a loop to compute the grand inventory value, and flag any item with quantity less than 5 as "Low Stock".

Code:

```
83 class Product:
84     def __init__(self, name: str, price: float, quantity: int):
85         self.name = name
86         self.price = price
87         self.quantity = quantity
88
89     def total_value(self) -> float:
90         """Calculates total value of the inventory item."""
91         return self.price * self.quantity
92
93
94 def manage_inventory(products: list):
95     """Processes product list, computes total value, and flags low stock."""
96     grand_total = 0.0
97
98     print(f"{'Product Name':<15} | {'Price':<10} | {'Quantity':<10} | {'Total Value':<12} | {'Status'}")
99     print("-" * 65)
100
101     for item in products:
102         item_val = item.total_value()
103         grand_total += item_val
104
105         # Check for low stock threshold (< 5)
106         status = "Low Stock" if item.quantity < 5 else "In Stock"
107
108         print(f"{item.name:<15} | ₹{item.price:<9.2f} | {item.quantity:<10} | ₹{item_val:<11.2f} | {status}")
109
110     print("-" * 65)
111     print(f"Grand Total Inventory Value: ₹{grand_total:,.2f}")
112
```

```
112
113
114 # Test Cases
115 if __name__ == "__main__":
116     inventory = [
117         Product("Wireless Mouse", 850.00, 12),
118         Product("Mechanical KB", 3200.00, 3), # Low stock
119         Product("USB-C Hub", 1500.00, 4),      # Low stock
120         Product("Monitor 24\"", 11000.00, 8)
121     ]
122
123     manage_inventory(inventory)
```

Output:

Product Name	Price	Quantity	Total Value	Status
Wireless Mouse	₹850.00	12	₹10200.00	In Stock
Mechanical KB	₹3200.00	3	₹9600.00	Low Stock
USB-C Hub	₹1500.00	4	₹6000.00	Low Stock
Monitor 24"	₹11000.00	8	₹88000.00	In Stock

Observation:

- ❑ **Key Finding:** Iterating over object lists correctly accumulates the grand inventory total ($\sum \text{price} \times \text{quantity}$).
- ❑ **Stock Alert:** Items with quantity < 5 are accurately flagged as "Low Stock".

Q4. Class + Loop + Nested Conditions – Electricity Bill

Scenario:

An electricity board wants to calculate bills based on units consumed.

Task:

Using AI-assisted code completion, create a Python class ElectricityBill that:

- Stores the consumer name and units consumed.
- Calculates the bill using:
 - First 100 units → ₹2/unit
 - 101–200 units → ₹3/unit
 - Above 200 units → ₹5/unit
- Uses appropriate **loops and conditional statements** where required.
- Displays the consumer name, units, and total bill.
- Create at least three objects and display their bills.
- Test the program with different unit values, including an edge value such as 100 or 200.

Prompt:

Write a Python program using an ElectricityBill class with attributes consumer_name and units. Implement slab-based billing logic: first 100 units at ₹2/unit, 101–200 units at ₹3/unit, and above 200 units at ₹5/unit. Instantiate at least 3 objects including boundary/edge test cases (e.g., 100 and 200 units) and display the bill details.

Code:

```

129
130 class ElectricityBill:
131     def __init__(self, consumer_name: str, units: float):
132         self.consumer_name = consumer_name
133         self.units = units
134
135     def calculate_bill(self) -> float:
136         """Calculates total electricity bill using slab pricing."""
137         units = self.units
138         if units <= 0:
139             return 0.0
140
141         bill = 0.0
142
143         # Tier 1: First 100 units @ ₹2/unit
144         if units <= 100:
145             bill = units * 2
146         # Tier 2: 101 to 200 units @ ₹3/unit
147         elif units <= 200:
148             bill = (100 * 2) + ((units - 100) * 3)
149         # Tier 3: Above 200 units @ ₹5/unit
150         else:
151             bill = (100 * 2) + (100 * 3) + ((units - 200) * 5)
152
153         return bill
154
155     def display_bill(self):
156         """Displays formatted bill summary."""
157         total_bill = self.calculate_bill()
158         print(f"Consumer Name : {self.consumer_name}")
159         print(f"Units Consumed: {self.units}")
160         print(f"Total Bill      : ₹{total_bill:,.2f}")
161         print("-" * 35)
162

```

```

162
163
164 # Test Cases
165 if __name__ == "__main__":
166     consumers = [
167         ElectricityBill("Amit Kumar", 100), # Edge Case: 100 units (First tier max)
168         ElectricityBill("Suresh Roy", 180), # Second tier (Between 101-200)
169         ElectricityBill("Rohan Singh", 200), # Edge Case: 200 units (Second tier max)
170         ElectricityBill("Ananya Sen", 320) # Third tier (Above 200)
171     ]
172
173     print("--- Electricity Bill Details ---\n")
174     for consumer in consumers:
175         consumer.display_bill()

```

Output:

```

--- Electricity Bill Details ---

Consumer Name : Amit Kumar
Units Consumed: 100
Total Bill    : ₹200.00
-----
Consumer Name : Suresh Roy
Units Consumed: 180
Total Bill    : ₹440.00
-----
Consumer Name : Rohan Singh
Units Consumed: 200
Total Bill    : ₹500.00
-----
Consumer Name : Ananya Sen
Units Consumed: 320
Total Bill    : ₹1,100.00
-----

```

Observation:

- ❑ **Key Finding:** Incremental slab calculations work accurately—charging ₹2/unit (1–100), ₹3/unit (101–200), and ₹5/unit (> 200).
- ❑ **Boundary Check:** Edge inputs at 100 and 200 units transition smoothly between price slabs without overcharging earlier tiers.